

Test Strategy

Objective

Prove the invariants, especially under concurrency. Prefer a small number of high-value integration tests against a real PostgreSQL instance over a large suite of mocked repository tests.

Each must-have acceptance criterion should link to at least one test ID. This is the verification stage of the compact spec-driven workflow; the specification is not considered satisfied merely because code was written.

Test layers

- Domain/unit tests: status transition rules and error mapping.
- Database integration tests: constraints, transactions, roster filters, and idempotency.
- API integration tests: request validation and response contracts.
- One optional UI smoke test: successful booking and visible status.

Required verification matrix

ID	Scenario	Setup	Expected result
T-01	Successful booking	Class has 0-2 confirmed	Booking confirmed; roster grows by one
T-02	Payment failure	Pending booking	<code>payment_failed</code> ; roster unchanged
T-03	Duplicate active booking	Existing pending booking	No second active row
T-04	Duplicate confirmed booking	Existing confirmed booking	No second confirmed row
T-05	Full class	Four confirmed	New booking not confirmed; roster remains four
T-06	Last-seat race	Three confirmed plus two pending	Exactly one confirmed; total four
T-07	Booking request replay	Same creation idempotency key	Same booking returned; one row
T-08	Payment result replay	Same payment idempotency key	Same outcome; one effective attempt
T-09	Conflicting payment replay	Same key, different payload	Stable conflict; state unchanged
T-10	Roster filtering	Mix of statuses	Only confirmed students returned

ID	Scenario	Setup	Expected result
T-11	Independent classes	Two simultaneous confirmations on different classes	Both can succeed without shared capacity interference
T-12	Invalid transition	Terminal booking gets new result	Rejected; terminal state preserved

Deterministic last-seat test

Avoid a race test that merely starts two requests "near" each other and hopes they overlap. Use two independent database connections and a barrier:

1. Seed a class with capacity 4 and exactly 3 confirmed bookings.
2. Create two pending bookings for different students.
3. Start both confirmation operations from separate connections.
4. Release both workers from a shared barrier.
5. Await both results.
6. Assert one result is `confirmed` and one is `capacity_unavailable`.
7. Query the database and assert confirmed count is exactly 4.
8. Repeat enough times to expose flaky transaction code, resetting data between runs.

The test should not assume which user wins.

Seed dataset

- Parent P1 with children S1 and S2.
- Parent P2 with children S3, S4, S5, and S6.
- Class C1: capacity 4, one confirmed booking; demonstrates ordinary availability.
- Class C2: capacity 4, exactly three confirmed bookings; demonstrates the final-seat race.
- Booking B-failed: `payment_failed`, excluded from roster.
- Booking B-duplicate-source: an active booking used by duplicate tests or created inside isolated fixtures.

Use deterministic IDs or expose seed labels so reviewers can understand fixtures. Tests should create their own data or reset the database to prevent order dependence.

Quality gates

Before submission:

1. Fresh clone setup succeeds from README commands.
2. Migrations and seed complete without manual database edits.
3. Type checking passes.
4. Linting passes.
5. All tests pass against PostgreSQL.
6. The race test passes repeatedly.

7. Roster never includes non-confirmed bookings.
8. README, AI_USAGE, and actual behavior agree.
9. No secrets or real child data exist in the repository.
10. Video walkthrough fits 5-8 minutes.

Saga and Kafka are not part of the take-home runtime, so tests should not mock or simulate infrastructure that does not exist. If a transactional outbox is added later, extend the matrix with atomic outbox creation, duplicate publication, consumer idempotency, retry, ordering, and reconciliation cases.

Manual demo checklist

- Show an available class and create a successful booking.
- Show the booking status and updated roster.
- Run a payment failure and prove the roster is unchanged.
- Attempt a duplicate booking and show the stable response.
- Run the automated final-seat race test and show the final count of four.